The finite regular-cover computation

Human Compression project

11 October 2026

This appendix prints the complete mathematical program used in the finite example of *Chirality kernels and the smallest regular cover*. It is copied without alteration from the independently reviewed computation `phase41_chiral_cover_check.g`. Its logical inputs are the three bare presentations, and its outputs establish all six group rows, the actual subgroup intersections, the mirror and covering maps, the local chirality subgroups, duality and the distinguishing word. GAP's exact algorithms are trusted; this is not a Lean certificate. The main theorem and infinite-family deductions do not depend on this finite computation.

The two old Python runners are preserved separately as operational provenance. Their manuscript-path assumptions are historical, so the current portable `check.py` executes the unchanged GAP program directly in a fresh directory and checks its complete expected output. Neither an old runner nor a finite validation model supplies an omitted all-parameter proof.

Complete required program

```
# All source presentations are literal. GAP's exact finitely presented/permutation
# group algorithms are retained; this is not a Lean kernel certificate.
SetInfoLevel(InfoWarning,0);;
check:=function(ok,label) if not ok then Print("FAIL ",label,"\n");QUIT_GAP(1);fi;end;;
F:=FreeGroup("a","b","c");; a:=F.1;; b:=F.2;; c:=F.3;;
base:=[(a*b)^2,(b*c)^2,(a*b*c)^2];;
prs:=rec(P:=Concatenation([a^6,b^9,c^6],base,[b*a^2*(a^2*b^4)^-1,c^2*b*(b^4*c^2)^-1]),T:=Con
catenation([a^6,b^9,c^2],base,[(a*b^-2)^2,a*b^2*a*(a*b^-1*a)^2]),Td:=Concatenation([a^2,b^9,
c^6],base,[(c^-1*b^2)^2,b*c^2*b*(c^-1*b*c^-1)^2]));;
make:=function(name)
 local fp,iso,g,s;
 fp:=F/prs.(name);iso:=IsomorphismPermGroup(fp);g:=Image(iso);
 check(IsBijective(iso),Concatenation(name," fp isomorphism"));
 s:=List(GeneratorsOfGroup(fp),x->Image(iso,x));
 check(ForAll(prs.(name),w->MappedWord(w,GeneratorsOfGroup(F),s)=One(g)),Concatenation(name,
 " literal relators"));
 return rec(g:=g,s:=s);
end;;
mix:=function(A,B)
 local d,e1,e2,s;
 d:=DirectProduct(A.g,B.g);e1:=Embedding(d,1);e2:=Embedding(d,2);
 s:=List([1..3],i->Image(e1,A.s[i])*Image(e2,B.s[i]));
 return rec(g:=Group(s),s:=s);
end;;
P:=make("P");;T:=make("T");;Td:=make("Td");;
Pm:=rec(g:=P.g,s:=[P.s[1]^-1,P.s[1]^2*P.s[2],P.s[3]]);;
R:=mix(P,Pm);;Q:=mix(R,T);;Qd:=mix(R,Td);;
expected:=rec(P:=[324,[6,9,6],[54,54],9,false],T:=[324,[6,9,2],[162,18],9,true],Td:=[324,[2,
9,6],[18,162],9,true],R:=[972,[6,9,6],[162,162],27,true],Q:=[2916,[6,9,6],[162,162],9,true],
Qd:=[2916,[6,9,6],[162,162],9,true]);;
A:=P;;
for name in ["P","T","Td","R","Q","Qd"] do
 A:=ValueGlobal(name);f:=Group(A.s{[1,2]});v:=Group(A.s{[2,3]});m:=Group([A.s[2]]);
 reg:=GroupHomomorphismByImages(A.g,A.g,A.s,[A.s[1]^-1,A.s[1]^2*A.s[2],A.s[3]]);
 row:=[Size(A.g),List(A.s,Order),[Size(f),Size(v)],Size(Intersection(f,v)),reg<>fail];
 check(row=expected.(name),Concatenation(name," exact row"));
 check(Size(m)=9,Concatenation(name," medial"));
 check(Intersection(Group([A.s[1]]),m)=TrivialSubgroup(A.g),Concatenation(name," facet
 IP"));
```

```
 check(Intersection(Group([A.s[3]]),m)=TrivialSubgroup(A.g),Concatenation(name," vertex
 IP"));
 check(ForAll(base,w->MappedWord(w,GeneratorsOfGroup(F),A.s)=One(A.g)),Concatenation(name,"
 rotation relations"));
 if reg<>fail then check(IsBijective(reg),Concatenation(name," mirror automorphism"));fi;
 if name<>"R" then check(Intersection(f,v)=m,Concatenation(name," end IP"));else
 check(Intersection(f,v)<>m,"R fails IP");fi;
 Print("ROW ",name," ",row,"\n");
od;
for pair in [[R,P],[Q,R],[Qd,R]] do
 cov:=GroupHomomorphismByImages(pair[1].g,pair[2].g,pair[1].s,pair[2].s);
 check(cov<>fail and IsSurjective(cov),"marked cover");
 check(Size(Kernel(cov))=3,"degree three");
od;
p1:=GroupHomomorphismByImages(R.g,P.g,R.s,P.s);;
p2:=GroupHomomorphismByImages(R.g,P.g,R.s,Pm.s);;
check(p1<>fail and p2<>fail,"two mirror projections");
Xf:=Image(p1,Intersection(Group(R.s{[1,2]}),Kernel(p2)));;
Xv:=Image(p1,Intersection(Group(R.s{[2,3]}),Kernel(p2)));;
check(Xf=Group([P.s[2]^3]) and Xv=Xf,"local chirality obstruction b^3");
Print("CHIRALITY facet=vertex=<b^3> ORDER 3\n");
faciso:=GroupHomomorphismByImages(Group(T.s{[1,2]}),Group(R.s{[1,2]}),T.s{[1,2]},R.s{[1,2]})
;;
check(faciso<>fail and IsBijective(faciso),"T facet is R facet");
dualiso:=GroupHomomorphismByImages(T.g,Td.g,T.s,[Td.s[3]^-1,Td.s[2]^-1,Td.s[1]^-1]);;
check(dualiso<>fail and IsBijective(dualiso),"Td is literal dual of T");
w:=(b*c^-1)^2*a*c^2*a^-1;;
check(List([Q,Qd],A->Order(MappedWord(w,GeneratorsOfGroup(F),A.s)))=[1,3],"nonisomorphism
witness");
Print("DISTINGUISH (b*c^-1)^2*a*c^2*a^-1 ORDERS [1,3]\n");
Print("PASS GAP ",GAPInfo.Version," six exact groups, all maps, IP, regularity, duality,
witness\n");
QUIT_GAP(0);
```

Finite diagnostics of the reflection formula

The next complete program checks the two-copy action in three cyclic groups and in the nonabelian symmetric group of degree three. Its countermodels explain the roles of inversion, the copy swap and generation. It is diagnostic evidence only; the quantified theorem is proved in the common paper and its actual action formula is checked in Lean.

```
#!/usr/bin/env python3
"""Finite diagnostics of the actual two-copy formula, not its general proof."""
from itertools import permutations
import json

def require(ok, message):
    if not ok:
        raise RuntimeError(message)

def check_model(name, elements, mul, inv, one, marks, theta):
    chambers = [(g, side) for g in elements for side in (False, True)]
    def neighbor(a, x):
        g, side = x
        return (mul(g, a if side else inv(a)), not side)
    def reflect(x):
        return (theta(x[0]), not x[1])
    def translate(g, x):
        return (mul(g, x[0]), x[1])
    require(all(theta(mul(g,h)) == mul(theta(g),theta(h))
                for g in elements for h in elements), name + ' automorphism')
    require(len({theta(g) for g in elements}) == len(elements), name + ' bijective')
    require(all(theta(a) == inv(a) for a in marks), name + ' marking')
    counts = {'neighbor_involution': 0, 'reflection_involution': 0,
              'color_preservation': 0, 'conjugation': 0}
    for x in chambers:
        require(reflect(reflect(x)) == x, name + ' reflection involution')
        counts['reflection_involution'] += 1
        for a in marks:
            require(neighbor(a,neighbor(a,x)) == x, name + ' neighbor involution')
```

```
            require(reflect(neighbor(a,x)) == neighbor(a,reflect(x)), name + ' color')
            counts['neighbor_involution'] += 1
            counts['color_preservation'] += 1
        for g in elements:
            require(reflect(translate(g,x)) == translate(theta(g),reflect(x)), name + '
            conjugation')
            counts['conjugation'] += 1
    return {'name':name,'order':len(elements),'checks':counts}

def main():
    reports=[]
    for n in (3,5,7):
        reports.append(check_model('cyclic_'+str(n),list(range(n)),
            lambda a,b,n=n:(a+b)%n,lambda a,n=n:(-a)%n,
            0,[0,1,2],lambda a,n=n:(-a)%n))
    elements=list(permutations(range(3)))
    def mul(a,b): return tuple(a[b[i]] for i in range(3))
    def inv(a): return tuple(a.index(i) for i in range(3))
    one=(0,1,2);a=(1,2,0);b=(1,0,2)
    reports.append(check_model('symmetric_3',elements,mul,inv,one,[one,a,b],
                               lambda g:mul(mul(b,g),b)))
    controls=[]
    def cyc_neighbor(n,a,x):
        g,side=x
        return ((g+(a if side else -a))%n,not side)
    def cyc_reflect(n,multiplier,x,swap=True):
        return ((multiplier*x[0])%n,not x[1] if swap else x[1])
    # Identity is an automorphism of C5 but does not invert its generator.
    x=(0,False)
    left=cyc_reflect(5,1,cyc_neighbor(5,1,x))
    right=cyc_neighbor(5,1,cyc_reflect(5,1,x))
    controls.append({'removed':'inversion_on_marks','modulus':5,
                     'left':left,'right':right,'detected':left!=right})
    # Inversion without the copy swap does not commute with color 1.
    left=cyc_reflect(5,-1,cyc_neighbor(5,1,x),swap=False)
    right=cyc_neighbor(5,1,cyc_reflect(5,-1,x,swap=False))
    controls.append({'removed':'swap_of_chamber_copies','modulus':5,
                     'left':left,'right':right,'detected':left!=right})
    # The mark 0 does not generate C7. Multiplication by 2 inverts that
    # mark but its square is not the identity automorphism.
    left=cyc_reflect(7,2,cyc_reflect(7,2,(1,False)))
    right=(1,False)
    controls.append({'removed':'generation','modulus':7,'marks':[0],
                     'automorphism_multiplier':2,'left':left,'right':right,
                     'detected':left!=right})
    require(all(c['detected'] for c in controls),'a hypothesis control failed')
    print(json.dumps({'status':'PASS','models':reports,'controls':controls},indent=2))

if __name__=='__main__':
    main()
```